CYED3 · INTEGRATIVE PROJECT · 2026-2

ResumeLens

Does a résumé satisfy the formal pattern of a professional profile?

Regular expressions | Transducers | Finite automata | CFG + textX

Johan Mora · Juan David Fajardo · Joshua Sayur

THE PROBLEM

One skill, many ways to write it

`JS · Javascript · ECMAScript`	→ `JAVASCRIPT`
`Postgres · Postgre SQL`	→ `POSTGRESQL`
`React.js · ReactJS`	→ `REACT`

Our goal

Decide, for four professional profiles, whether the qualifications a résumé states explicitly satisfy a formal pattern.

No ranking and no scores: every decision is explicit and testable.

ARCHITECTURE

One pipeline, four formal tools

1

Regular expressions

Extraction

Name, contact, experience, education and raw skills.

2

Transducers

Normalization

Every spelling becomes one canonical token.

`Postgres → POSTGRESQL`

3

Finite automata

Classification

ACCEPTED or REJECTED for each of the four profiles.

4

CFG + textX

Profile language

A validated program, rendered as HTML and Markdown.

One general solution for all four profiles. Each stage has one responsibility; the pipeline only orchestrates.

STAGE 1 · REGULAR EXPRESSIONS

Regex finds text; it never decides

Name and contact
Capitalized name line · e-mail · phone

Experience
3 years of experience · 3 años de experiencia

Education
Bachelor's degree · Computer Science

Skills
Comma-separated items under “Technical Skills:”

INPUT

```
Technical Skills:
JS, React.js, NodeJS, Postgres, Git.
```

OUTPUT

```
["JS", "React.js", "NodeJS",
"Postgres", "Git"]
```

E-MAIL PATTERN

```
[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
```

The output keeps the original spelling. Deciding that “JS” means JavaScript is the next stage’s job.

STAGE 2 · FINITE-STATE TRANSDUCERS

Many spellings in, one token out

Nine transducers

One per family of aliases (Node, React, version control…)

One path per alias

ε on the intermediate letters, the canonical token on the last one

Case-insensitive

Two transitions per letter, built into δ

Idempotent

Normalizing a canonical token returns the same token

FORMAL DEFINITION

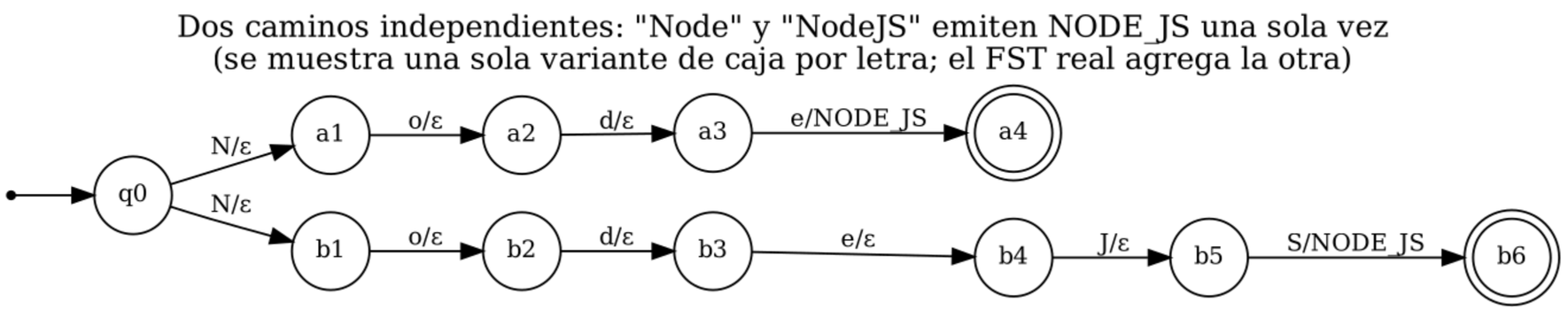
$$M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$$

SIZE OF THE FULL SET

791 states
1,511 transitions
35 canonical skills

STAGE 2 · EXAMPLE

Node and NodeJS emit NODE_JS once



The paths share their prefix. The token appears only at the end of a complete alias, so a prefix never produces output by mistake.

STAGE 3 · FINITE AUTOMATA

The state remembers what is covered

State = covered mandatory categories

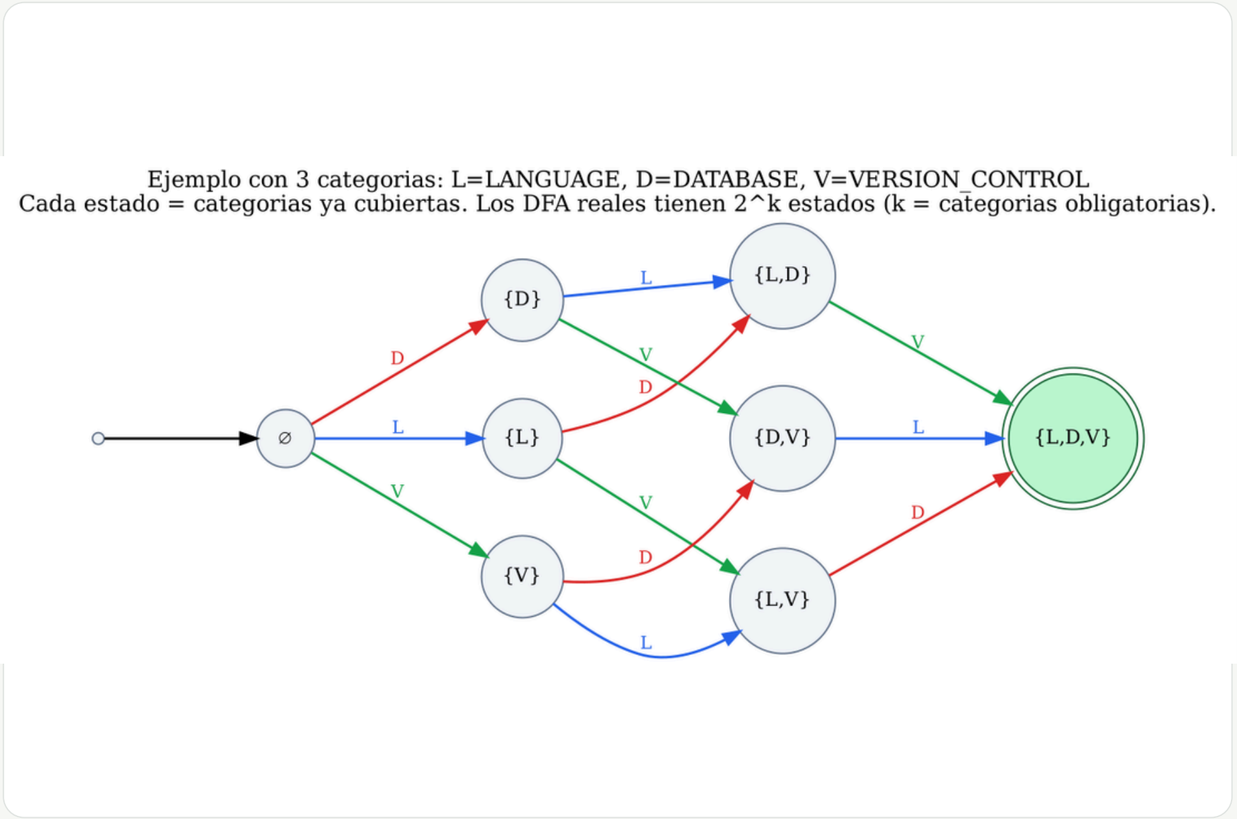
With k mandatory categories there are 2^k states.

Accepting state = all covered

Optional categories are self-loops; the order does not matter.

Partial and minimal

A missing transition means rejection; no DFA can be smaller.



STAGE 3 · FOUR PROFILES

Same method, four different patterns

PROFILE	STATES	ALPHABET
Full Stack Developer	32	14
Machine Learning Engineer	64	10
QA Engineer	64	13
Data Analyst	16	11

Context matters. SQL is a DATABASE skill for Full Stack and ML, but a QUERY_LANGUAGE skill for Data Analyst.

Tokens foreign to a profile are dropped before its automaton, so they never cause a rejection.

STAGE 4 · GRAMMAR AND TEXTX

The result is a validated program

CANDIDATE PROFILE LANGUAGE

```
candidate "Wednesday Addams" {
  experience { years: 3 }
  skills { JAVASCRIPT, REACT,
    NODE_JS, POSTGRESQL, GIT }
  classification {
    FULL_STACK_DEVELOPER: ACCEPTED
    QA_ENGINEER: REJECTED …
  }
}
```

Abridged: contact, education and the other results are omitted.

Syntax: textX

A context-free grammar in a .tx file builds the model

Semantic checks

Non-empty name · years between 0 and 60 · no duplicate skills · one result per profile

Two outputs

The valid program is rendered as HTML and Markdown

LIVE DEMO

From pasted text to a validated program

1
Paste a résumé
Press “Analyze”.

2
Follow the stages
Extraction, normalization, four decisions.

3
Edit the program
Set years: 99 and watch it be rejected.

4
Try the edge cases
An empty résumé and a QA résumé with COBOL.

```
streamlit run src/ui/app.py
```

VALIDATION

345 automated tests, all passing

114	93	83	35	20
Normalization	Classification	Grammar	Extraction	Pipeline

ORDER DOES NOT MATTER

For Data Analyst, all 24 permutations of the mandatory skills are accepted.

EACH STAGE ON ITS OWN

Normalization, classification, grammar and extraction are tested separately, then end to end.

CONCLUSIONS

What it does, and what it does not

Strengths

Every decision is explicit and testable.

Each stage has one responsibility.

One general method for four profiles.

Limits

Only the skills in the dictionary are recognized.

It checks stated qualifications, not real ability.

It supports a human review; it does not replace it.

The use of AI tools is documented in docs/ai-usage.md. Thank you. Questions?